BAZE DE DATE

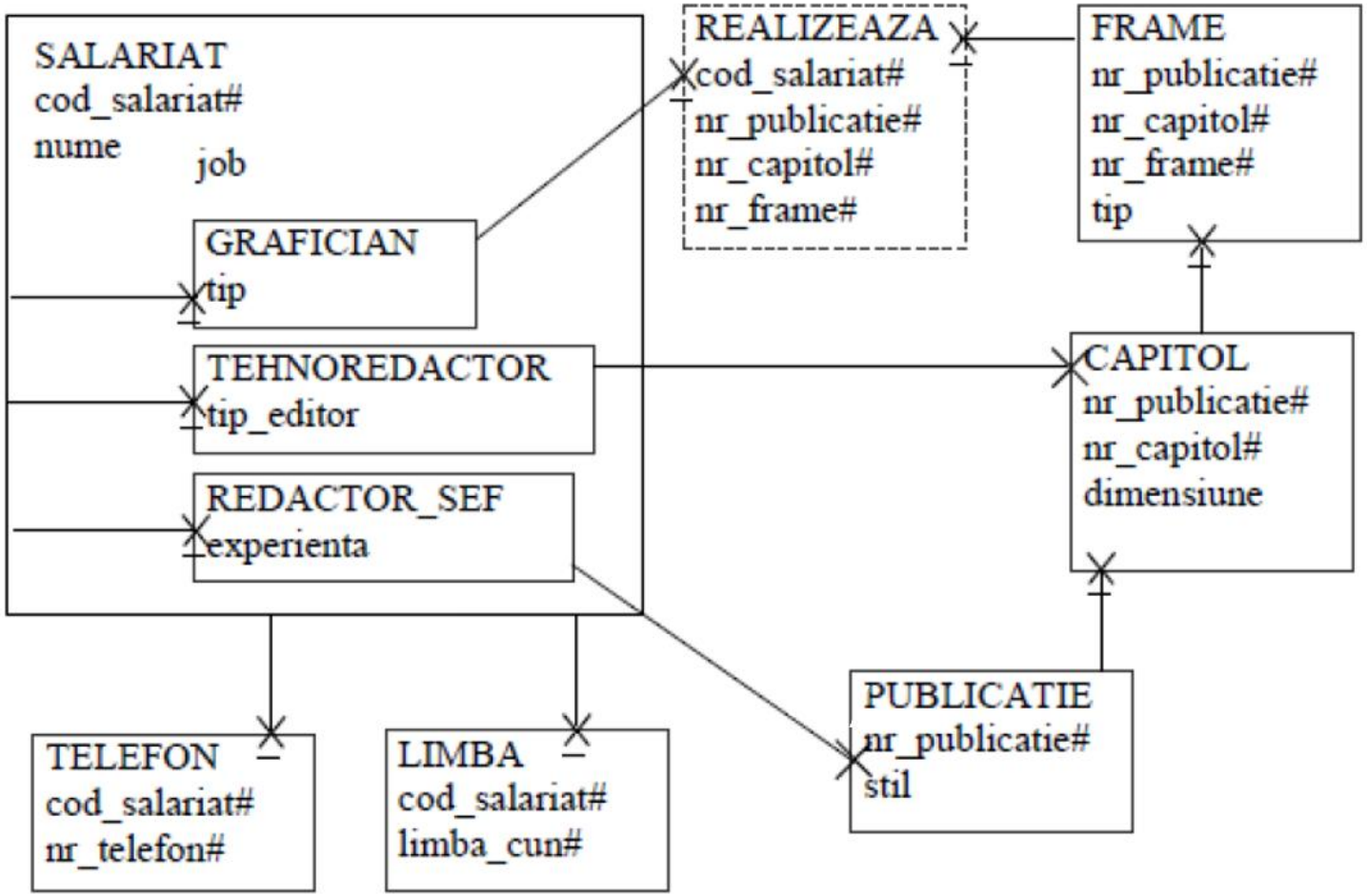
CURS 9

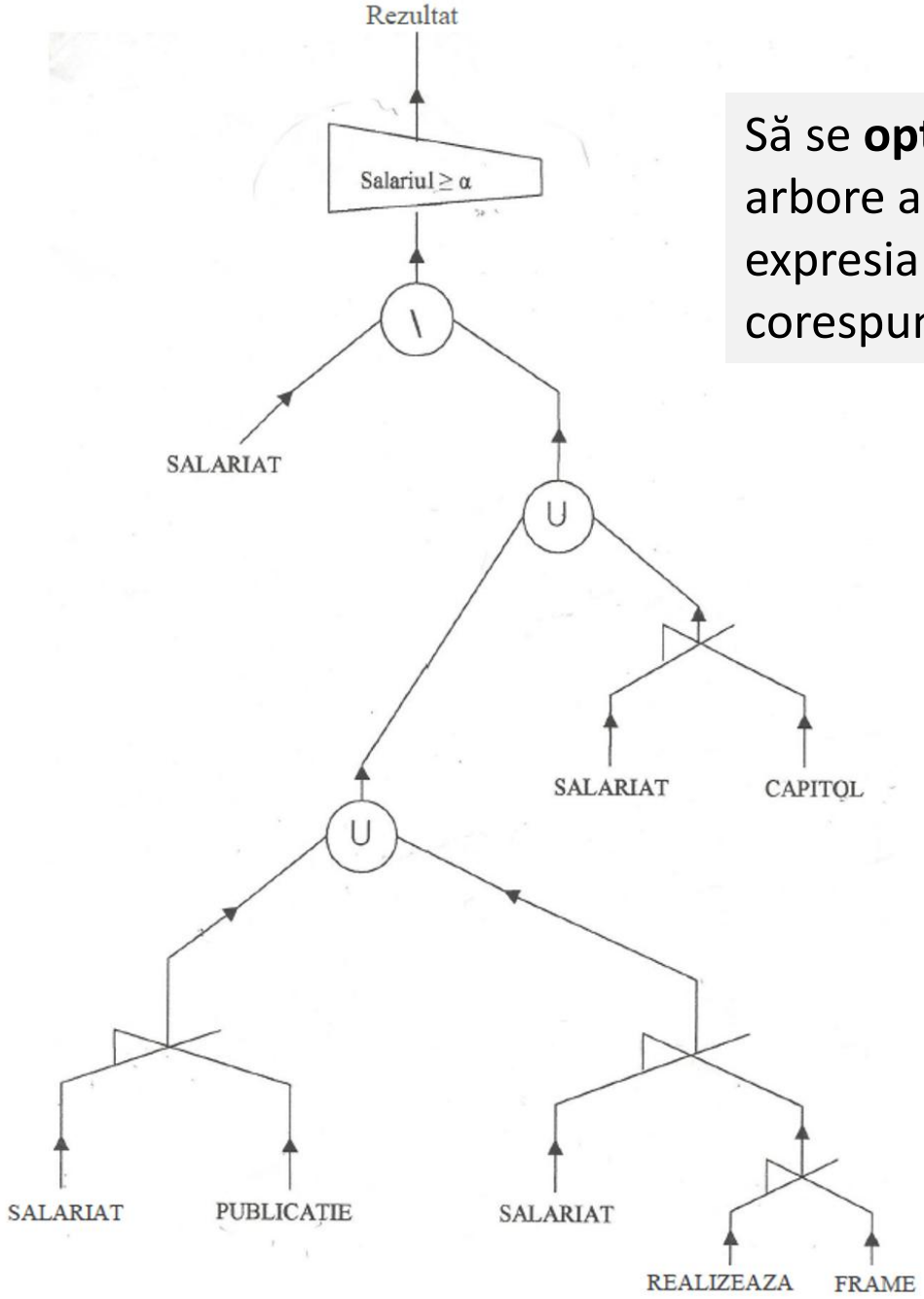
Exerciții Arbori

Rezolvările exercițiilor din cadrul acestui curs se află și pe site, în documentul numit “**Exercitii_Arbori**”

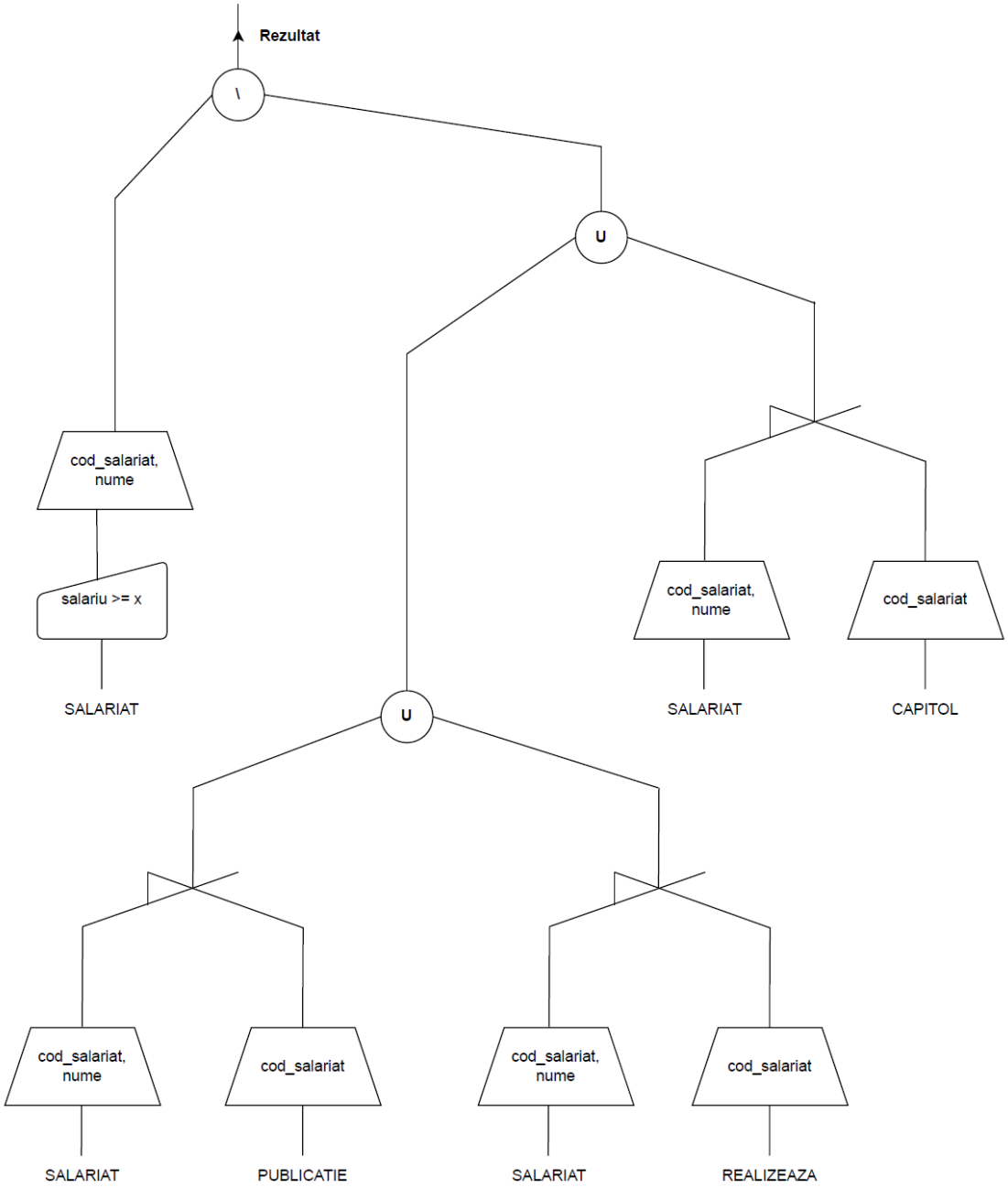
Exercițiul 1

Să se afișeze informații despre salariații care **nu** contribuie la machetarea niciunei publicații, dar au salariul mai mare decât o valoare data **– cerere în limbaj natural**





Să se **optimizeze** următorul arbore algebric și să se scrie expresia algebrică corespunzătoare


Rezultat
\
∪
cod_salariat, nume
salariu >= x
SALARIAT
cod_salariat, nume
cod_salariat
SALARIAT
CAPITOL
∪
cod_salariat, nume
cod_salariat
cod_salariat, nume
cod_salariat
SALARIAT
PUBLICATIE
SALARIAT
REALIZEAZA

Expresia algebrică:

R1 = PROJECT(REALIZEAZA, cod_salariat)
R2 = PROJECT(SALARIAT, cod_salariat, nume)
R3 = SEMIJOIN(R1, R2, cod_salariat)
R4 = PROJECT(PUBLICATIE, cod_salariat)
R5 = SEMIJOIN(R4, R2, cod_salariat)
R6 = UNION(R3, R5)
R7 = PROJECT(CAPITOL, cod_salariat)
R8 = SEMIJOIN(R7, R2, cod_salariat)
R9 = UNION(R8, R6)
R10 = SELECT(SALARIAT, salariu >= x)
R11 = PROJECT(R10, cod_salariat, nume)
R12 = MINUS(R11, R9)
Rezultat = PROJECT(R12, cod_salariat, nume)

Exercițiul 2

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **l1** și **l2**, iar tipul contractantului este '**investiție**'.

Scrieți **cererea SQL**, urmată de **expresia algebrică** și **arborele algebric**.

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **l1** și **l2**, iar tipul contractantului este '**investiție**'.

Cerere SQL?

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **l1** și **l2**, iar tipul contractantului este ‘**investiție**’.

SELECT cod_contractant, telefon, val_investitie, durata

FROM contractant co JOIN contract ct ON(co.cod_contractant = ct.cod_contractant)

WHERE val_investitie > l1 AND val_investitie < l2 AND

tip_contractant = 'investitie';

```
SELECT cod_contractant, telefon, val_investitie, durata
FROM contractant co JOIN contract ct ON(co.cod_contractant = ct.cod_contractant)
WHERE val_investitie > l1 AND val_investitie < l2 AND
        tip_contractant = 'investitie';
```

Expresia algebrică - neoptimă:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

R5 = SELECT(R4, tip_contractant = ‘investitie’)

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrică:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

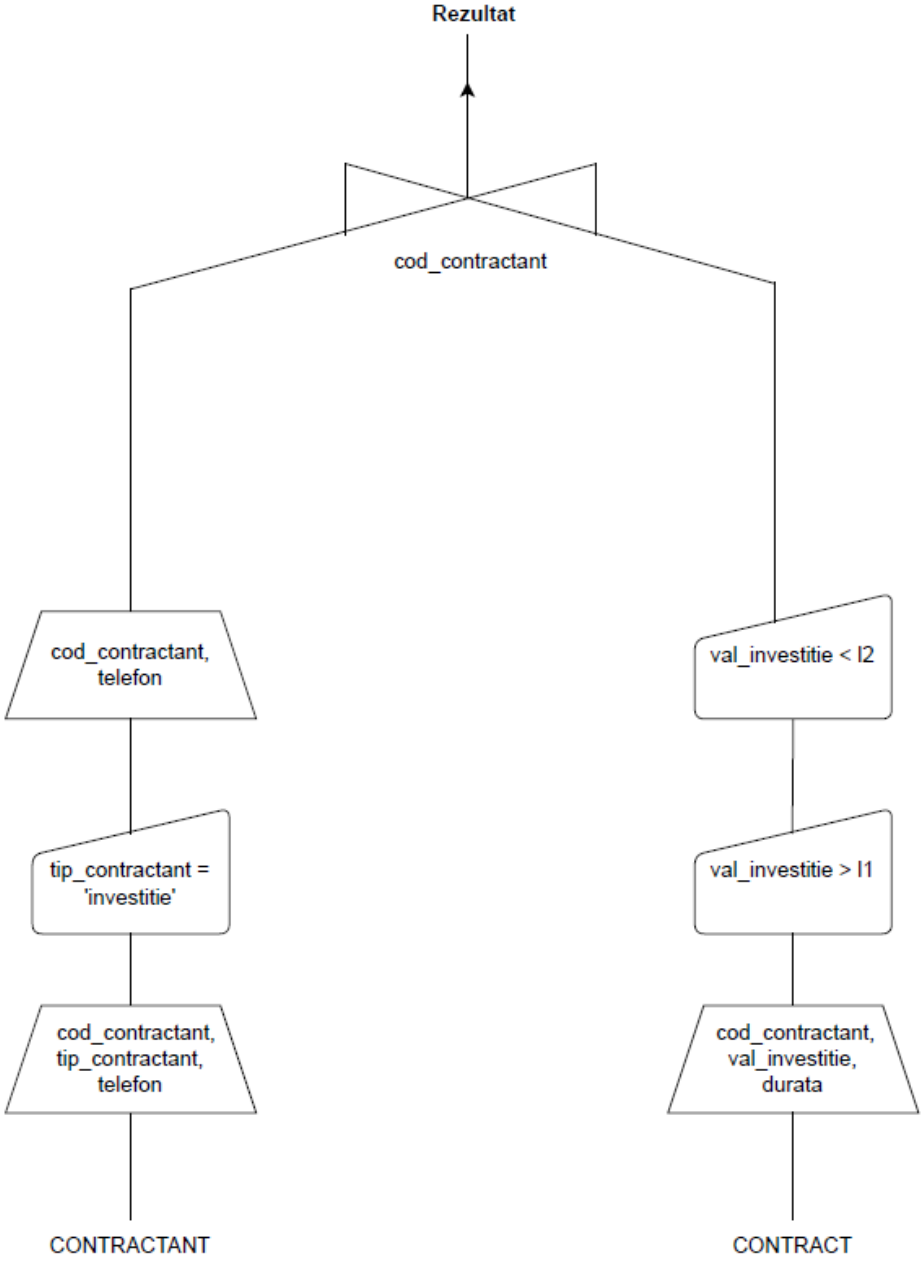
R5 = SELECT(R4, tip_contractant = ‘investitie’)

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Join-ul din relația **Rezultat** returnează toate coloanele pe care le au relațiile primite ca argument.

Arborele algebric - neoptimizat:



Expresia algebrică - neoptimizată:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

R5 = SELECT(R4, tip_contractant = ‘investitie’)

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrica - optimizată:

R1 = SELECT(CONTRACT, val_investitie > l1 AND val_investitie < l2)

R2 = PROJECT(R1, cod_contractant, val_investitie, durata)

R3 = SELECT(CONTRACTANT, tip_contractant = ‘investitie’)

R4 = PROJECT(R3, cod_contractant, telefon)

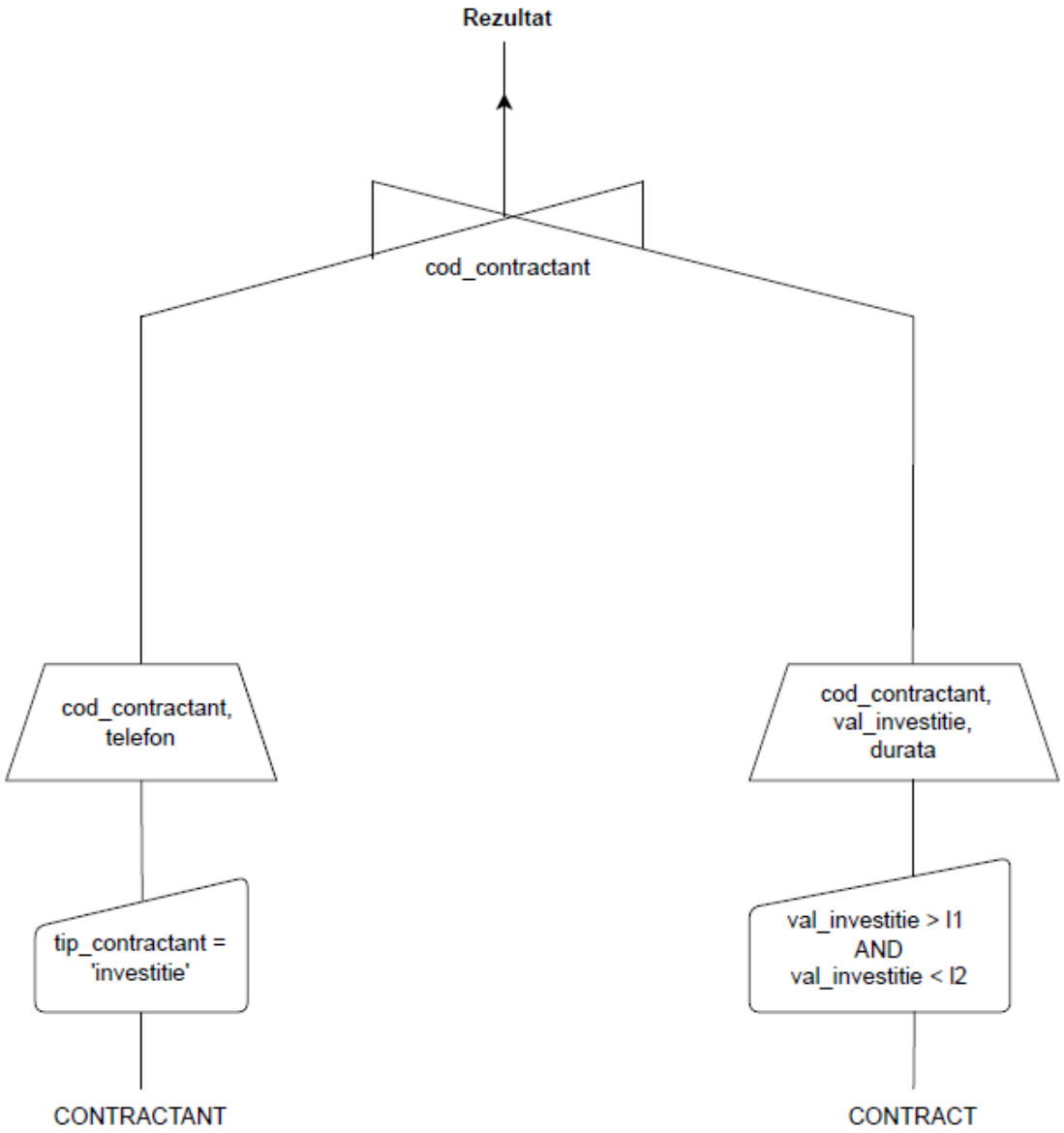
Rezultat = JOIN(R4, R2, cod_contractant) -> **rezultat optimizat**

Explicațiile detaliate și pașii optimizării se află în documentul **Exerciții_Arbori** - de pe site.

În același mod se procedează și la **EXAMENUL FINAL**.

Arborele algebric

– optimizat:



De ce învățăm expresii și arbori algebrici?

- SQL-ul este doar o limbă declarativă – noi spunem "ce vrem", dar nu "cum" să se execute.
- Sistemul de gestiune a bazei de date (ex: Oracle Database, PostgreSQL, MySQL) transformă automat cererea SQL într-un arbore algebric (intern, în memorie).
- Optimizatorul bazei de date aplică reguli matematice de rescriere ca să transforme arborele într-o formă mai rapidă și mai eficientă de executat.
- Prin urmare, dacă știm să lucrăm cu expresii și arbori algebrici, știm cum “gândește” sistemul și putem scrie de la început cereri SQL mai eficiente.
- Mai eficient înseamnă **timp de răspuns mai mic**, **încărcare mai mică** pe server, **costuri mai mici**.

Cum legăm teoria de codul real?

- În SQL Developer facem **EXPLAIN PLAN** → vedem arborele de execuție (fiecare nod e un operator algebric: SELECT, JOIN, PROJECT etc.)

- Dacă cererea SQL nu este optim scrisă, arborele generat e mai lung și implică scanări inutile de date, join-uri prea multe și lente sau chiar filtrări târzii (după ce am adus datele, în loc să filtrăm la început).

- Dacă scriem cereri mai bine gândite (optimizate logic), arborele e mai scurt și mai eficient - filtrăm mai repede, procesăm mai puține date, facem join mai mic.

Exemplu concret pe diagrama HR - neoptimizat

Să se afișeze numele, prenumele angajaților, dar și denumirea departamentului în care aceștia lucrează. Se vor afișa doar acei angajați care au salariul mai mare decât 5000 și care lucrează în departamentele aflate în locația cu id-ul 1700.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e JOIN departments d
                        ON e.department_id = d.department_id
WHERE e.salary > 5000 AND d.location_id = 1700;
```

Expresie algebrică - neoptimizată:

R1 = JOIN(EMPLOYEES, DEPARTMENTS, department_id)
R2 = SELECT(R1, salary > 5000 AND location_id = 1700)
R3 = PROJECT(R2, first_name, last_name, department_name)

Problema:
Se realizează JOIN-ul între toți angajații și toate departamentele, după care se filtrează salariile și locațiile ⇒ se procesează prea multe date.

Exemplu concret pe diagrama HR - optimizat

Să se afișeze numele, prenumele angajaților, dar și denumirea departamentului în care aceștia lucrează. Se vor afișa doar acei angajați care au salariul mai mare decât 5000 și care lucrează în departamentele aflate în locația cu id-ul 1700.

```
SELECT e.first_name, e.last_name, d.department_name
FROM (SELECT * FROM employees WHERE salary > 5000) e
JOIN (SELECT * FROM departments WHERE location_id = 1700) d
ON e.department_id = d.department_id;
```

Expresie algebrică - optimizată:

R1 = SELECT(EMPLOYEES, salary > 5000)
R2 = SELECT(DEPARTMENTS, location_id = 1700)
R3 = JOIN(R1, R2, department_id)
R4 = PROJECT(R3, first_name, last_name, department_name)

De ce este mai optimă această variantă?

- Se realizează filtrarea **înaintea** JOINULUI ⇒ mai puține date combinate ⇒ JOIN mai mic și mai rapid.
- Se reduce încărcarea serverului.

Cum arată în planul de execuție (EXPLAIN PLAN)

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			57	6
MERGE JOIN			57	6
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID	21	2
Filter Predicates				
LOCATION_ID=1700				
INDEX	DEPT_ID_PK	FULL SCAN	27	1
SORT		JOIN	57	4
Access Predicates				
EMPLOYEES.DEPARTMENT_ID=DEPARTMENTS.DEPARTMENT_ID				
Filter Predicates				
EMPLOYEES.DEPARTMENT_ID=DEPARTMENTS.DEPARTMENT_ID				
TABLE ACCESS	EMPLOYEES	FULL	57	3
Filter Predicates				
SALARY>5000				

Ambele cereri au același plan de execuție. De ce?

De ce ambele cereri au același plan de execuție

- **Motivul principal** este că optimizatorul SQL (Oracle Database, SQL Server, PostgreSQL etc.) este foarte inteligent.

- Chiar dacă se implementează o cerere „neoptimizată”, **optimizatorul o rescrie automat** în forma cea mai bună, înainte să o execute.

În Oracle SQL Developer:

- Optimizatorul aplică automat **Predicate Pushdown** și **View Merging –** adică extrage condițiile din subcereri sau le împinge cât mai aproape de sursa de date, **chiar dacă acest lucru nu a fost făcut manual**.

- Indiferent dacă filtrările sunt scrise după join sau înainte, Oracle mută singur filtrările cât mai devreme.

Concluzii

- La nivel de învățare, este important să știm cum gândește optimizatorul.
- Chiar dacă un SGBD modern optimizează singur, în practică nu toate bazele de date sunt la fel de bune. În baze de date mai simple sau în cazuri speciale, scrierea corectă face diferența.
- În interviuri tehnice sau proiecte mari (enterprise), este important să cunoaștem ce face optimizatorul și să implementăm cereri clare și predictibile.
- Chiar dacă SQL Developer arată același plan în cazuri mai simple, în sisteme reale, cu date mari și tabele complexe, modul în care scriem SQL-ul poate influența mult performanța.
- Expresiile și arborii algebrici ne ajută să înțelegem cum “gândește” sistemul și să scriem cereri optime de la început.
- În cazuri complexe, precum bazele de date mari sau query-uri pe sute de milioane de rânduri, scrierea corectă poate duce la diferențe de 10x sau 100x în performanță.

Regulile lui Codd

Avantajele modelului relaţional:

- fundamentare matematică riguroasă;
- independenţă fizică a datelor;
- posibilitatea filtrărilor;
- existenţa unor structuri de date simple;
- realizarea unei redundanţe minime;
- supleţe în comunicarea cu utilizatorul neinformatician;

Regulile lui Codd

Limite ale modelului relaţional:

- rămâne totuşi redundanţă;
- nu lucrează cu obiecte complexe;
- nu realizează gestiunea totala a datelor distribuite;

Regulile lui Codd

- În anul 1985, E.F. Codd a publicat un set de **12 reguli** în raport cu care un sistem de gestiune a bazelor de date poate fi apreciat ca relaţional
  - Niciun sistem de gestiune a bazelor de date nu respectă absolut toate regulile definite de Codd, dar acest lucru nu împiedică etichetarea acestor sisteme drept relaţionale

- Nu trebuie apreciat un SGBD ca fiind relaţional sau nu, ci **măsura în care acesta este relaţional**, deci numărul regulilor lui Codd pe care le respectă

Regulile lui Codd

- **Regula 1 – regula reprezentării informaţiei**. *Într-o bază de date relaţională, informaţia este reprezentată la nivel logic sub forma unor* ***tabele ce poartă numele de relaţii***.
- **Regula 2 – regula accesului garantat la date**. *Fiecare valoare dintr-o bază de date relaţională trebuie să poată fi accesată în mod logic printr-o* ***combinaţie formată din numele relaţiei, valoarea cheii primare şi numele atributului***.
- **Regula 3 – regula reprezentării informaţiei necunoscute**. *Un sistem relaţional trebuie să permită utilizatorului definirea unui tip de date numit „**null**“ pentru reprezentarea unei informaţii necunoscute la momentul respectiv.*

Regulile lui Codd

- **Regula 4 – regula dicţionarelor de date**. *Asupra descrierii bazelor de date* (*informaţii relative la relaţii, vizualizări, indecşi etc.*) *trebuie să se poată aplica* ***aceleaşi operaţii*** *ca şi asupra datelor din baza de date*.
- **Regula 5 – regula limbajului de interogare**. *Trebuie să existe cel puţin un* ***limbaj pentru prelucrarea bazei de date***.
- **Regula 6 – regula de actualizare a vizualizării**. *Un SGBD trebuie să poată determina dacă o vizualizare poate fi actualizată şi să stocheze rezultatul interogării într-un dicţionar de tipul unui catalog de sistem*.
- **Regula 7 – regula limbajului de nivel înalt**. *Regulile de prelucrare asupra unei relaţii luată ca întreg sunt valabile atât pentru operaţiile de regăsire a datelor, cât şi asupra operaţiilor de inserare, actualizare şi ştergere a datelor*.

Regulile lui Codd

- **Regula 8 – regula independenţei fizice a datelor**: *Programele de aplicaţie şi activităţile utilizatorilor nu depind de modul de depunere a datelor sau de modul de acces la date*.
- **Regula 9 – regula independenţei logice a datelor**. *Programele de aplicație trebuie să funcționeze normal chiar dacă structura logică a datelor se schimbă (de exemplu: adăugăm sau eliminăm un câmp dintr-un tabel).*
- **Regula 10 – regula independenţei datelor privind integritatea acestora**. ***Regulile de integritate trebuie să fie definite într-un sublimbaj relaţional, nu în programul de aplicaţie*.**

Regulile lui Codd

- **Regula 11 – regula independenţei datelor din punct de vedere al distribuirii**. *Distribuirea datelor pe mai multe calculatoare dintr-o reţea de comunicaţii de date nu trebuie să afecteze programele de aplicaţie*.
- **Regula 12 – regula versiunii procedurale a unui SGBD**. *Orice componentă procedurală a unui SGBD trebuie să respecte aceleaşi restricţii de integritate ca şi componenta relaţională.*

Regulile lui Codd

Deoarece regulile lui Codd sunt prea severe pentru a fi respectate de un SGBD operaţional, s-au formulat **criterii minimale de definire a unui sistem de gestiune relational**:

- Un SGBD este **minimal relaţional** dacă:
  - toate datele din cadrul bazei sunt reprezentate prin **valori în tabele**;
  - nu există pointeri observabili de către utilizator;
  - sistemul suportă operatorii relaţionali de **proiecţie, selecţie şi compunere naturală**, fără limitări impuse din considerente interne;
- Un SGBD este **complet relaţional** dacă este minimal relaţional şi satisface în plus condiţiile:
  - sistemul suportă **restricţiile de integritate de bază** (unicitatea cheii primare, constrângerile referenţiale, integritatea entităţii);
  - sistemul suportă **toate operaţiile de bază ale algebrei relaţionale**;

Normalizarea Relațiilor

- În procesul modelării unei baze de date relaţionale, o etapă importantă o reprezintă **normalizarea relaţiilor conceptuale**, adică obţinerea de relaţii „moleculare“ fără a pierde nimic din informaţie, pentru a elimina:
  - **redundanţa**;
  - **anomaliile reactualizării informaţiilor;**

Normalizarea Relațiilor

- Tehnica normalizării permite:
  - obţinerea unei **scheme conceptuale rafinate** printr-un proces de **ameliorare progresivă** a unei scheme conceptuale iniţiale a bazei de date relaţionale;
- După fiecare etapă de ameliorare, relaţiile bazei de date ating un anumit **grad de perfecţiune**, deci se află într-o anumită **formă normală**
  - Trecerea unei relaţii dintr-o formă normală în alta, presupune **eliminarea unui anumit tip de dependență** nedorită, care este transformată în dependenţă admisibilă, adică dependenţă care nu provoacă anomalii;

Normalizarea Relațiilor

Procesul de ameliorare a schemei conceptuale trebuie:

- să garanteze **conservarea datelor**, adică în schema conceptuală finală trebuie să figureze toate datele din cadrul schemei iniţiale;
- să garanteze **conservarea dependenţelor** dintre date, adică în schema finală fiecare dependenţă trebuie să aibă determinantul şi determinatul în schema aceleiaşi relaţii;
- să reprezinte o **descompunere minimală** a relaţiilor iniţiale, adică niciuna dintre relaţiile care compun schema finală nu trebuie să fie conţinută într-o altă relaţie din această schema;

Normalizarea Relațiilor

Există două metode pentru a modela **baze de date relaţionale fără anomalii sau pierderi de informaţie**:

- **Schema descompunerii** pleacă de la o schemă relaţională universală ce conţine toate atributele BD.
  - Schema se descompune prin **proiecţii succesive** în subrelaţii;
  - Descompunerea se opreşte când continuarea ei ar duce la pierderi de informaţie;
  - Algoritmii de descompunere se bazează, în general, pe descrierea formală a dependenţei dintre atribute;

Normalizarea Relațiilor

- **Schema sintezei** pleacă de la o mulţime de atribute independente.
  - Utilizând proprietăţi de semantică şi legături între atribute se pot compune noi relaţii, astfel încât, acestea să nu sufere de anumite anomalii;
  - Algoritmii se bazează, în general, pe teoria grafurilor pentru a reprezenta legăturile dintre atribute;

Dependenţe funcţionale

- O **relaţie universală** este o relaţie ce grupează toate atributele care modelează sistemul real cercetat

- Fie *E*, **mulţimea dependenţelor** considerate de proiectantul bazei de date pentru o schemă relaţională sau pentru o relaţie universală.

  - Plecând de la o mulţime de proprietăţi formale ale dependenţelor, proprietăţi considerate drept reguli de deducţie (**axiome**), poate fi obţinută **mulţimea maximală de dependenţe asociate lui *E***. Această mulţime defineşte **închiderea lui *E***.

Dependenţe funcţionale

- Fie E mulţimea dependenţelor unei relaţii şi $p_1, p_2, ..., p_r$, $r \geq 1$, proprietăţi formale ale acestor dependenţe.
  - Dacă există o mulţime E', astfel încât orice dependenţă a mulţimii E este derivabilă din E' prin aplicarea proprietăţilor $p_1, p_2, ..., p_r$, atunci mulţimea E' defineşte **acoperirea** lui E pentru proprietăţile $p_1, p_2, ..., p_r$.

- E' este **o acoperire minimală** pentru E, dacă nu există nicio submulţime proprie, nevidă a lui E' care să fie o acoperire pentru E. **Evident, E şi E' au închideri identice, deci dispun de acelaşi potenţial informaţional!**

Dependenţe funcţionale

- Fie $R(A_1, A_2, ..., A_n)$ o schemă relaţională şi fie X, Y submulţimi de atribute ale lui R.
  - X **determină funcţional** Y sau Y depinde funcţional (FD) de X, dacă pentru orice relaţie r (valoare curentă a lui R) **nu există două tupluri care să aibă aceleaşi valori pentru atributele lui *X* şi să aibă valori diferite pentru cel puţin un atribut din *Y*.** Cu alte cuvinte, o valoare a lui X, determină unic o valoare a lui Y.

- Notaţie: $X \rightarrow Y$; X este numit **determinant,** iar Y este numit **determinat** (sau dependent).
  - Dependenţa funcţională $X \rightarrow Y$ este **trivială** dacă $Y \subseteq X$.

Dependenţe funcţionale

În momentul proiectării și modelării unei baze de date relaționale, pot să apară anomalii. Procesul de optimizare a relațiilor conceptuale poartă numele de **normalizare**. Pentru a depista eventualele redundanțe se utilizează **dependențele funcționale**.

Fie **R (A1, A2, … , An)** o schemă relațională.

De exemplu:
EMPLOYEES(employee_id, first_name, salary, job_id, job_title)

EMPLOYEES

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Dependenţe funcţionale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Tipuri de anomalii:

1) **Redundanța –** stocarea repetată a unor valori (în exemplul anterior IT_PROG și PROGRAMATOR).
2) **Anomalii de actualizare –** modificarea valorii redundante de la 1) poate duce la modificarea valorii doar pentru unele înregistrări, și nu pentru toate.
3) **Anomalii de inserare –** nu se poate insera un job (job_id, job_title) decât dacă se inserează, în baza de date, și un angajat asociat.
4) **Anomalii de ștergere –** dacă se sterge un angajat, se pierd automat și datele despre job.

Dependenţe funcţionale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Apar aceste anomalii deoarece în relația **EMPLOYEES** sunt atribute care aparțin unor mulțimi diferite X si Y și deci unor relații diferite.

X -> Y - **X determină funcțional pe Y sau Y depinde funcțional de X** dacă pentru orice relație **nu există** două tupluri care să aibă aceleași valori pentru atributele lui X și să aibă valori diferite pentru cel puțin un atribut din Y. Cu alte cuvinte, o valoare a lui X **determină unic** o valoare a lui Y

De exemplu:
100 KING 24000 IT_PROG PROGRAMATOR
101 JOHN 9500 MGR MANAGER

Dependenţe funcţionale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

De exemplu:
100 KING 24000 IT_PROG PROGRAMATOR
101 JOHN 9500 MGR MANAGER

IT_PROG determină funcțional PROGRAMATOR

Deci, o valoare a lui X determină unic o valoare a lui Y. Astfel, se obțin următoarele submulțimi de atribute ale relației EMPLOYEES:

{job_id} -> {job_title} – job_id determină funcțional pe job_title

Dependenţe funcţionale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observă următoarele submulțimi de atribute:

X’ = {employee_id, first_name, salary, job_id}
Y’ = {job_id, job_title}

Din submulțimile X’ și Y’ de atribute, se obțin următoarele scheme relaționale:

EMPLOYEES(employee_id, first_name, salary, job_id)
JOBS(job_id, job_title)

Dependenţe funcţionale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observă că informația din relația **EMPLOYEES** trebuie, de fapt, stocată în două relații, astfel:

EMPLOYEES

emp_id	first_name	salary	Job_id
100	KING	24000	IT_PROG
101	JOHN	9500	MGR
102	HUNOLD	17000	IT_PROG

JOBS

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

Dependenţe funcţionale

Acest procedeu se numește **descompunerea relațiilor**.

Descompunerile se realizează pe baza formelor normale pe care le vom studia în cursul următor, iar definirea formelor normale se bazează pe noțiunea de ***dependență funcțională***.

Dependenţe funcţionale

EMPLOYEES

emp_id	first_name	salary
100	KING	24000
101	JOHN	9500
102	HUNOLD	17000

JOBS

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

În exemplul anterior, există următoarele dependențe funcționale:

employee_id -> first_name, salary (se citește: employee_id **determină funcțional** first_name și salary sau first_name și salary **depind funcțional** de employee_id)

job_id -> job_title

Dependenţe funcţionale

Pornind de la aceste dependențe, putem afirma că:

- Dacă doi angajați ar avea același employee_id – înseamnă că este același angajat;
- Dacă doi angajați au același job_id – atunci rezultă că au, implicit, și același job_title;

Dependenţe funcţionale

- Comparând toate submulţimile de atribute ale unei relaţii şi determinând legăturile dintre ele, se pot obţine toate dependenţele funcţionale pe care o relaţie le satisface.
  - Această abordare **nu** este eficientă, consumând mult timp;

- Există posibilitatea ca, ştiind **anumite dependenţe** funcţionale şi utilizând **reguli de deducţie**, să fie obţinute **toate** dependenţele funcţionale.

Dependenţe funcţionale

Fie X, Y, Z, W mulţimi de atribute ale unei scheme relaţionale R şi fie următoarele axiome:

- **Ax1 – reflexivitate.** $X \rightarrow X$. Mai general, dacă $Y \subseteq X$, atunci $X \rightarrow Y$.
- **Ax2 – creşterea determinantului (augmentare).** Pot fi considerate următoarele formulări echivalente pentru această axiomă.
  - Dacă $X \rightarrow Y$ şi $X \subseteq Z$, atunci $Z \rightarrow Y$. ($\subseteq$ -> fiecare element din X este si element in Z; X este inclus in Z)
  - Dacă $X \rightarrow Y$ şi $W \subseteq Z$, atunci $X \cup Z \rightarrow Y \cup W$.
  - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y \cup Z$.
  - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y$.

  In urma reuniunii se obtin dependente functionale valide
- **Ax3 – tranzitivitate.** Dacă $X \rightarrow Y$ şi $Y \rightarrow Z$, atunci $X \rightarrow Z$.

Dependenţe funcţionale

- O mulţime de axiome este **completă** dacă şi numai dacă plecând de la o mulţime de dependenţe *E* se pot obţine toate dependenţele închiderii lui *E*, utilizând axiomele mulţimii.
- O mulţime de axiome este **închisă** dacă şi numai dacă plecând de la o mulţime de dependenţe *E*, nu poate fi dedusă cu ajutorul axiomelor o dependenţă care nu aparţine închiderii lui *E*.
- **Ullman** a demonstrat că axiomele Ax1 – Ax3, numite **axiomele lui Amstrong**, **reprezintă o mulţime închisă şi completă de axiome**.

Dependenţe funcţionale

- Nu toate dependenţele funcţionale sunt folositoare pentru modelarea relaţională.
- O dependenţă funcţională $X \rightarrow Y$ se numeşte **dependenţă funcţională totală** (FT), dacă şi numai dacă nu există nicio submulţime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$.
  - Dacă există o submulţime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$, atunci dependenţa funcţională $X \rightarrow Y$ este parţială. În axioma Ax2, dependenţa $Z \rightarrow Y$ este o dependenţă funcţională parţială;

Dependenţe funcţionale

- În cazul dependenţei funcţionale totale, axiomele lui Amstrong se reduc la o axiomă unică şi anume **pseudo-tranzitivitatea**:
  - dacă $X \rightarrow Y$ şi $W \cup Y \rightarrow Z$, atunci $W \cup X \rightarrow Z$

- Această axiomă este o regulă de deducţie completă pentru total dependenţe:
  - pseudo-tranzitivitatea implică tranzitivitatea (daca $W = \emptyset$);
  - reflexivitatea nu poate fi utilizată pentru a obţine dependenţe totale;

Dependenţe funcţionale

- Dacă *F* este o mulţime de dependenţe funcţionale totale, atunci **închiderea pseudo-tranzitivă** F^+ a acestei mulţimi este reuniunea mulţimilor dependenţelor funcţionale totale care pot fi obţinute din *F* folosind axioma de pseudo-tranzitivitate. Multimea tuturor dependentelor functionale care se pot deduce din F se numeste inchiderea multimii de dependente F, notata cu F^+
- Două mulţimi de dependenţe funcţionale totale sunt **echivalente** dacă au **închideri pseudo-tranzitive identice**

Dependenţe funcţionale

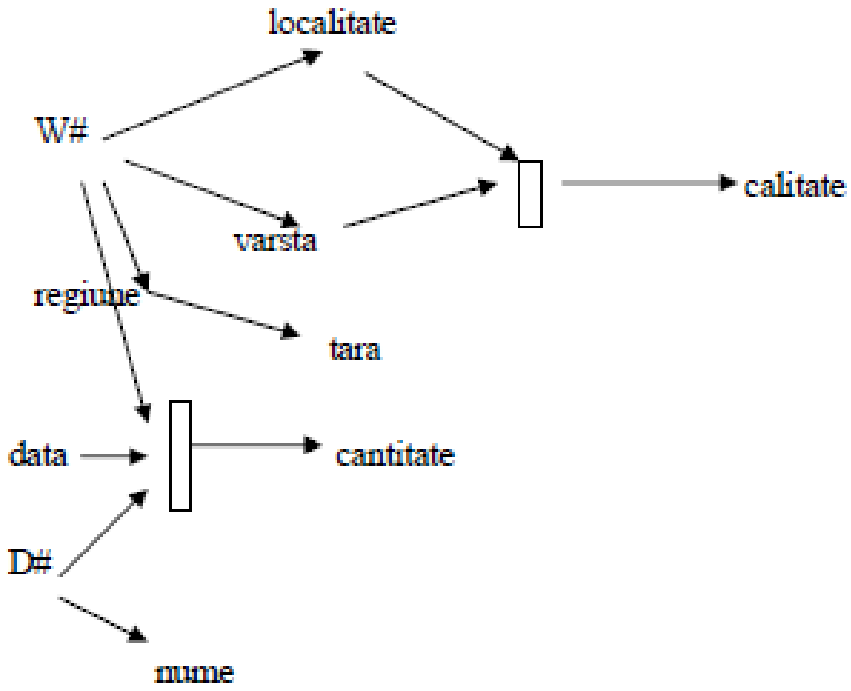
- Dependenţele funcţionale între atributele bazei pot fi reprezentate grafic
  - Fie $A = \{A_1, A_2, ..., A_n\}$ o mulţime de atribute şi fie o mulţime de dependenţe funcţionale $\{X_i \rightarrow A_j\}$, unde X_i este o submulţime a lui A.
- **Graful dependenţelor funcţionale** este un **graf direcţionat bipartit**, definit astfel:

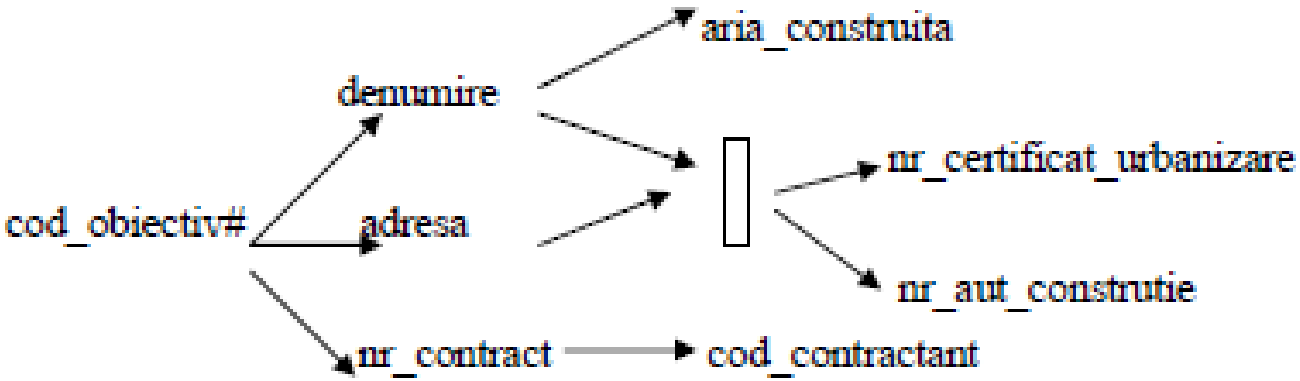
1. pentru fiecare atribut A_j există un singur **nod** având eticheta A_j;
2. pentru fiecare dependenţă funcţională de forma $A_i \rightarrow A_j$, există un **arc** de la A_i la A_j;
3. pentru fiecare dependenţă funcţională de forma $X_i \rightarrow A_j$, unde mulţimea X_i este definită de $X_i = \{Ai_1, ..., Ai_p\}$ cu $p > 1$, există un **nod auxiliar** etichetat prin X_i şi **o mulţime de arce** plecând de la $Ai_1, ..., Ai_p$ pentru a obţine pe X_i şi **printr-un arc adiţional** de la X_i la A_j. Nodurile X_i se reprezintă prin dreptunghiuri.

Dependenţe funcţionale

Exemple:

1. Graful dependenţelor funcţionale pentru schema relaţională **CONSUMATOR_DE_VIN**(W#, localitate, varsta, calitate, regiune, tara, D#, nume, data, cantitate).



Dependenţe funcţionale

2. Graful dependenţelor funcţionale pentru schema relaţională **OBIECTIV_INVESTITIE**. Dependentele sunt deduse din regulile impuse de beneficiar.